

DAT300

Backpropagation

Computational Graph Motivation

Modern machine learning models require efficient computation of functions and their derivatives. Training algorithms such as **gradient descent rely on gradients of the loss function**.

Why Computational Graphs?

- Simplify complex functions by breaking them into small operations.
- Enable systematic gradient computation using the chain rule.
- Form the basis of automatic differentiation and backpropagation in neural networks.
- Derivatives tell us how much the loss changes when a parameter change

If $\frac{dL}{dw} > 0$ increasing the **weight** increases the **loss** so the weight is decreased, and if $\frac{dL}{dw} < 0$ increasing the **weight** decreases the **loss** so the weight is increased (via gradient descent $w_{new} = w - \eta \frac{dL}{dw}$).

Derivatives of Activation functions

Function Type	Equation	Derivative
Linear	$f(x) = ax + c$	$f'(x) = a$
Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x) (1 - f(x))$
TanH	$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ReLU	$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parametric ReLU	$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
ELU	$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

Derivatives

Chain Rule

$$F(x) = f(g(h(u(v(x)))))$$

$$\frac{dF}{dx} = \frac{d}{dx}F(x) = \frac{d}{dx}[f(g(h(u(v(x)))))] = \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}$$

GIVEN

$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

(net input of the hidden layer)

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$$

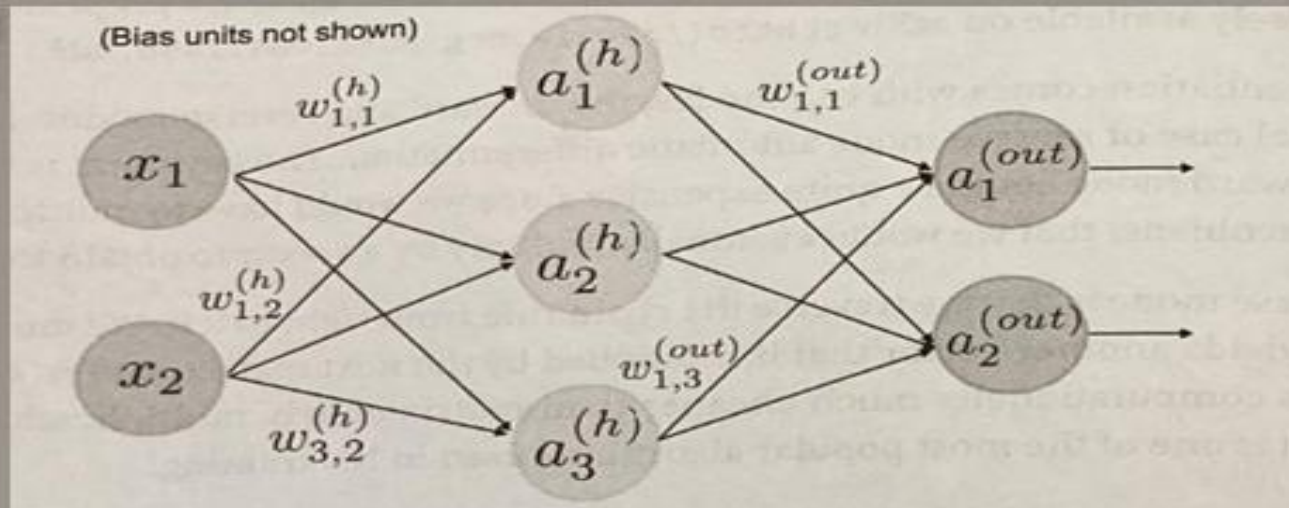
(activation of the hidden layer)

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

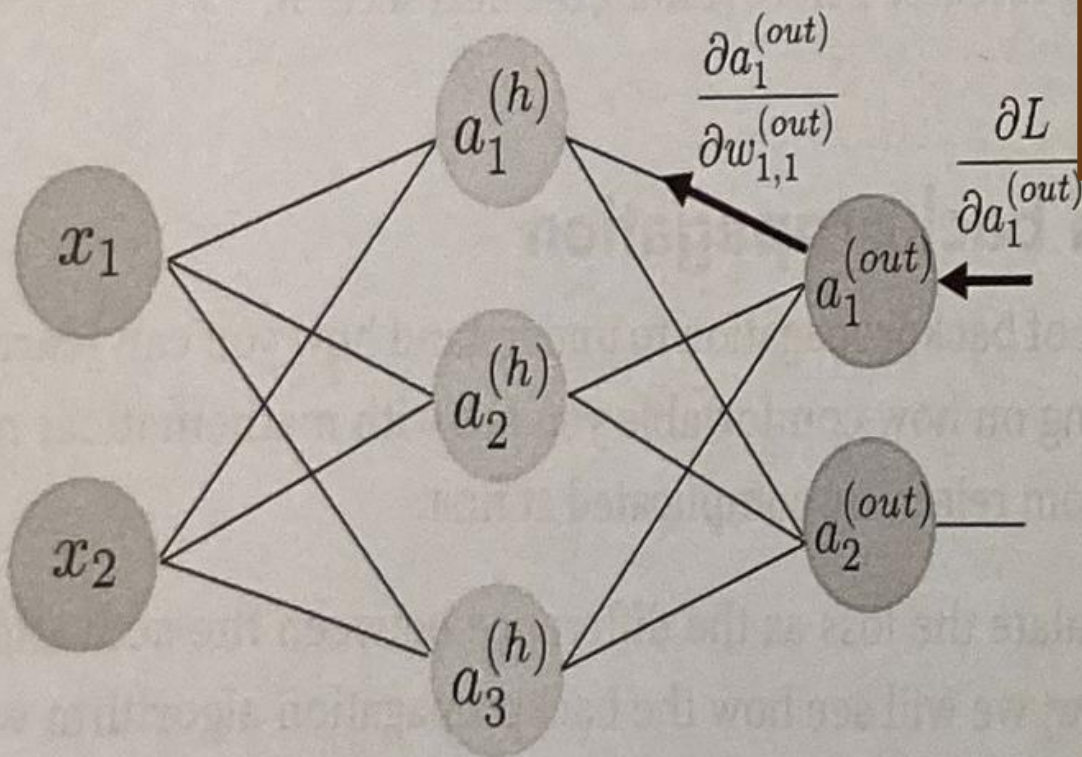
(net input of the output layer)

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

(activation of the output layer)



$$L(\mathbf{W}, \mathbf{b}) = \frac{1}{n} \sum_{i=1}^n \frac{1}{t} \sum_{j=1}^t (y_j^{[i]} - a_j^{(out)[i]})^2$$



GIVEN:

$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$$

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

Let's start with $\frac{\partial L}{\partial a_1^{(out)}}$, which is the partial derivative of the MSE loss (which simplifies to the squared error if we omit the mini-batch dimension) with respect to the predicted output score of the first output node:

$$\frac{\partial L}{\partial a_1^{(out)}} = \frac{\partial}{\partial a_1^{(out)}} (y_1 - a_1^{(out)})^2 = 2(a_1^{(out)} - y)$$

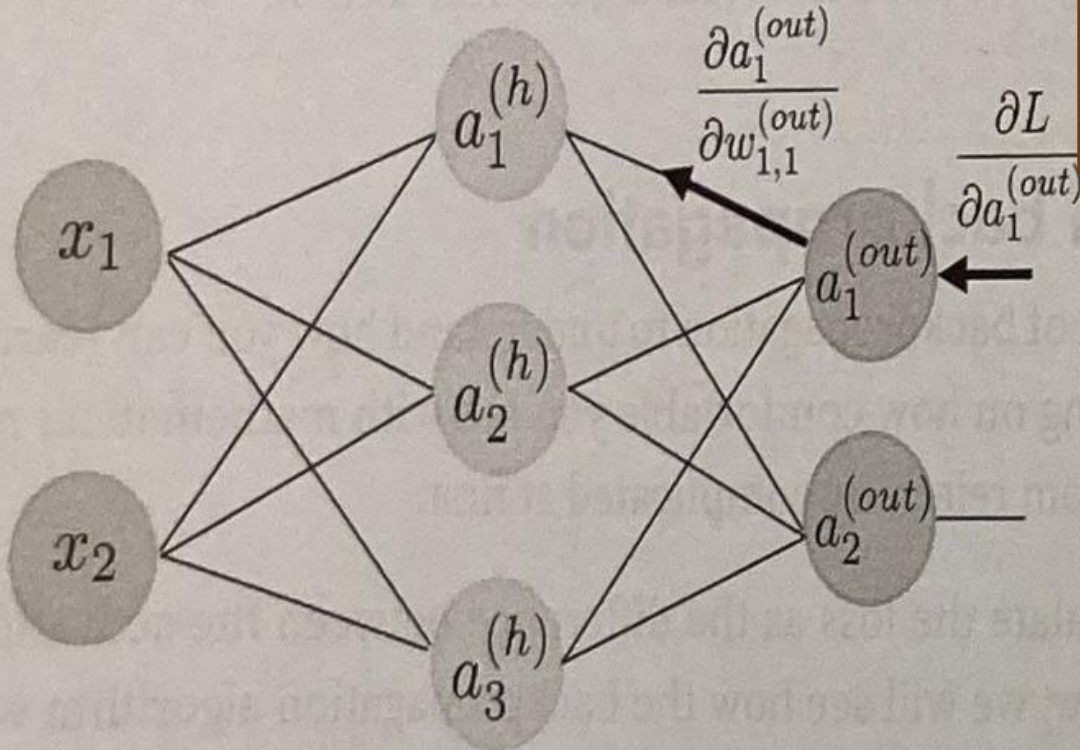
Python
Machine
Learning

Machine Learning and Deep Learning with Python, 3rd Edition, and TensorFlow 2

Third Edition - Includes TensorFlow 2, GANs, and Reinforcement Learning

Sebastian Raschka
& Vahid Mirjalili

Packt



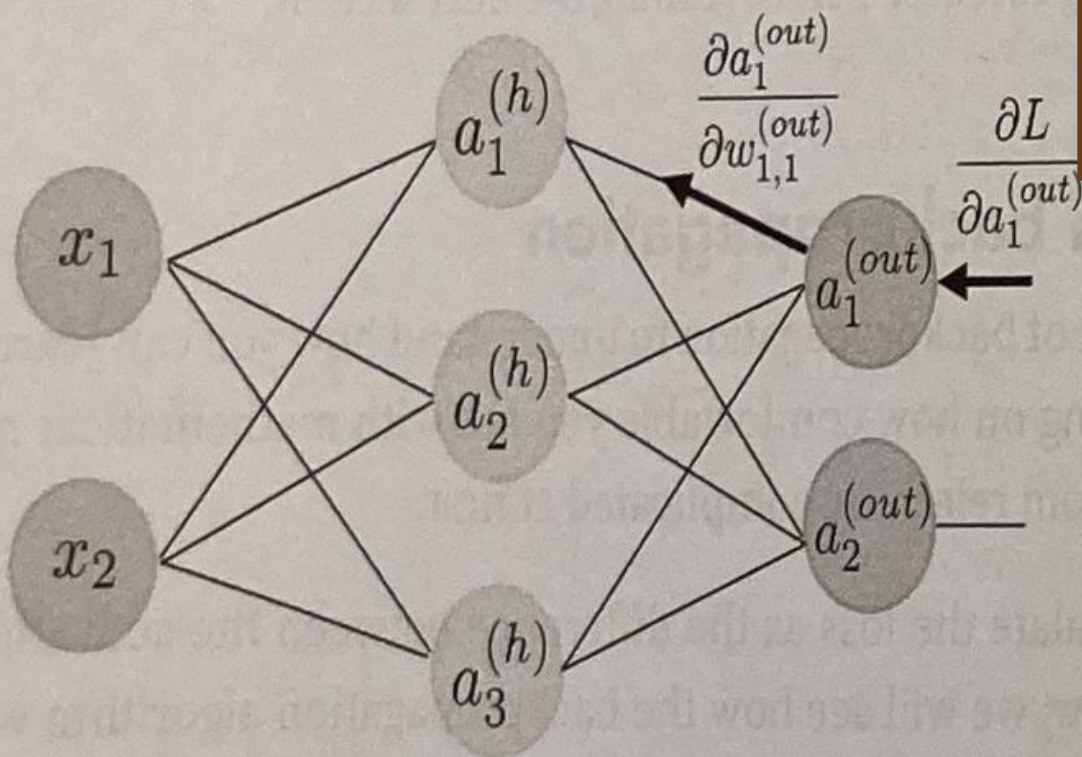
$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

GIVEN: $\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$



GIVEN:

$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$$

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

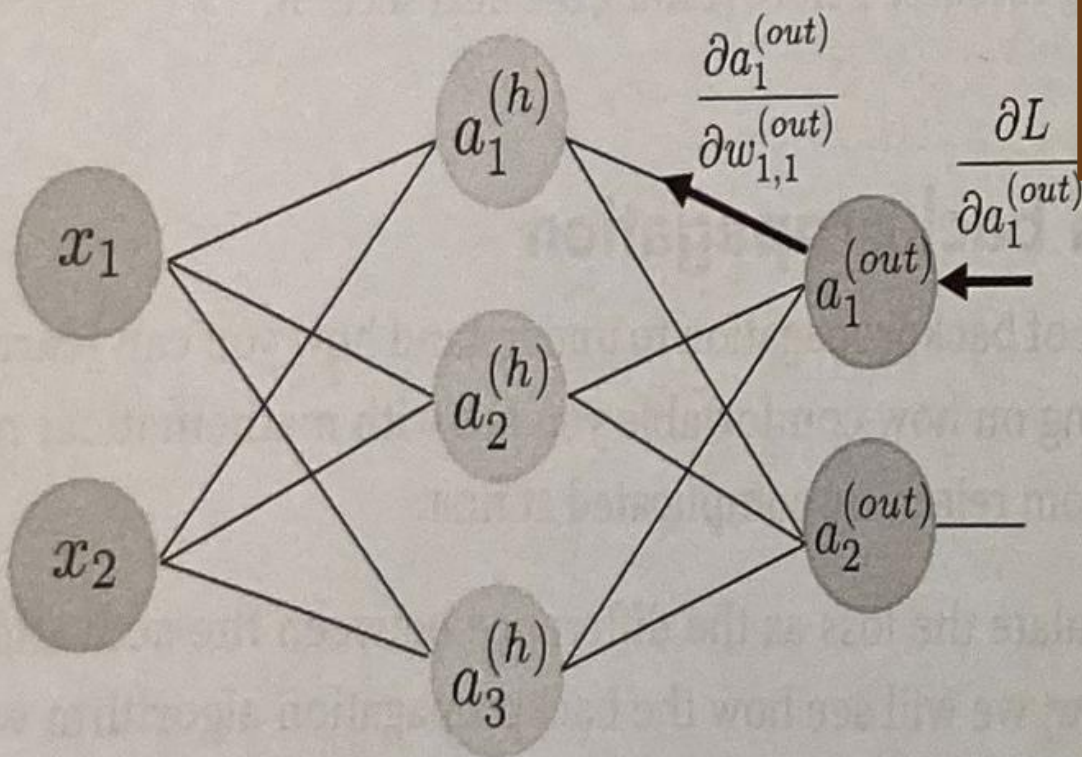
Python
Machine
Learning

Machine Learning and Deep Learning
with Python, scikit-learn, and TensorFlow 2

Third Edition - Includes TensorFlow 2,
GANs, and Reinforcement Learning

Sebastian Raschka
& Vahid Mirjalili

Packt



$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

GIVEN:

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$$

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

EXPERT INSIGHT

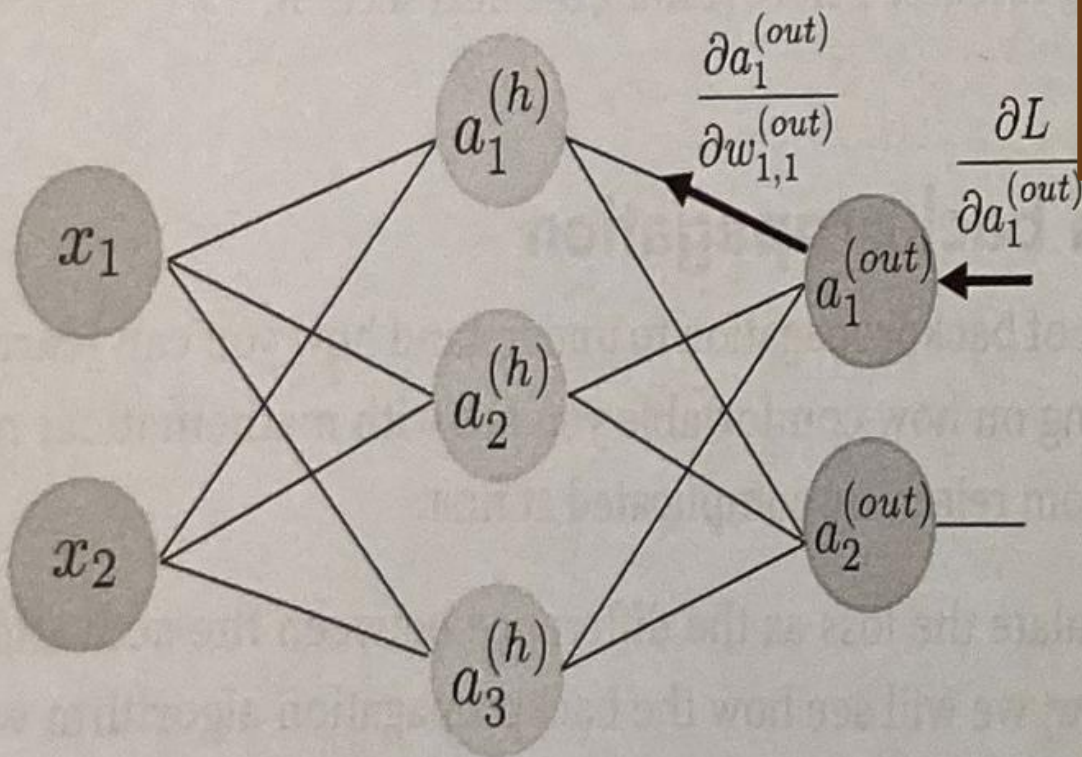
Python Machine Learning

Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2

Third Edition - Includes TensorFlow 2, GANs, and Reinforcement Learning

Sebastian Raschka
& Vahid Mirjalili

Packt



GIVEN:

$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$$

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

Let's start with $\frac{\partial L}{\partial a_1^{(out)}}$, which is the partial derivative of the MSE loss (which simplifies to the squared error if we omit the mini-batch dimension) with respect to the predicted output score of the first output node:

$$\frac{\partial L}{\partial a_1^{(out)}} = \frac{\partial}{\partial a_1^{(out)}} (y_1 - a_1^{(out)})^2 = 2(a_1^{(out)} - y)$$

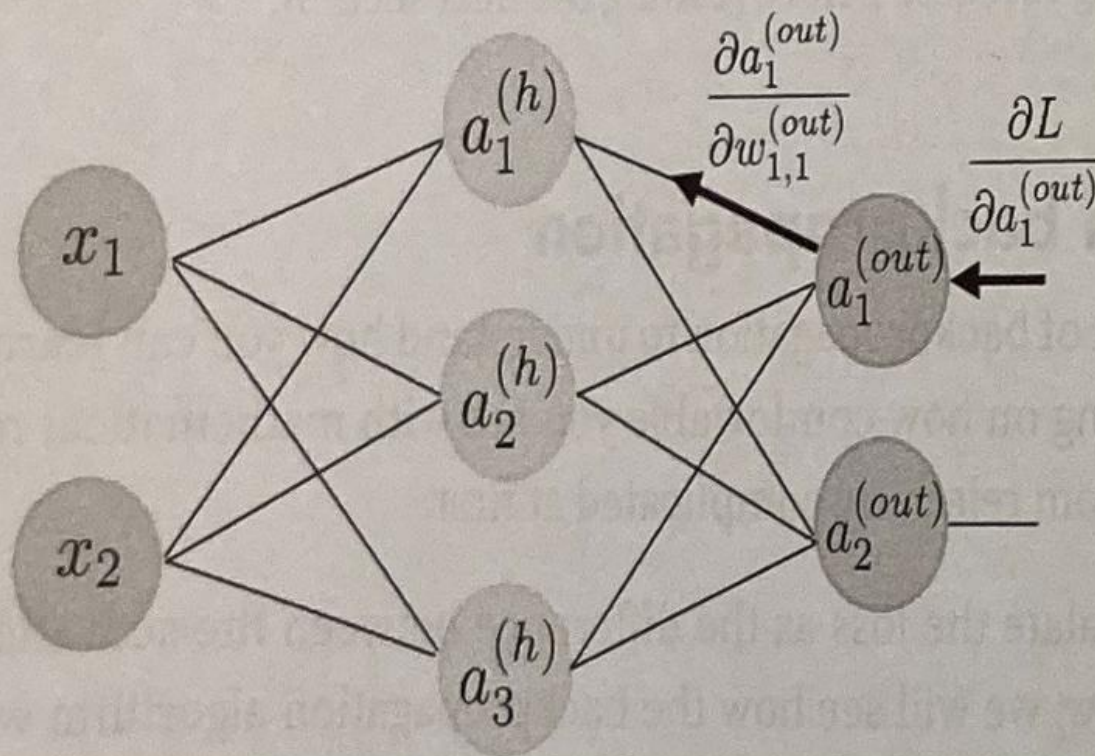
Python
Machine
Learning

Machine Learning and Deep Learning with Python, 3rd Edition, and TensorFlow 2

Third Edition - Includes TensorFlow 2, GANs, and Reinforcement Learning

Sebastian Raschka
& Vahid Mirjalili

Packt



GIVEN:

$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$$

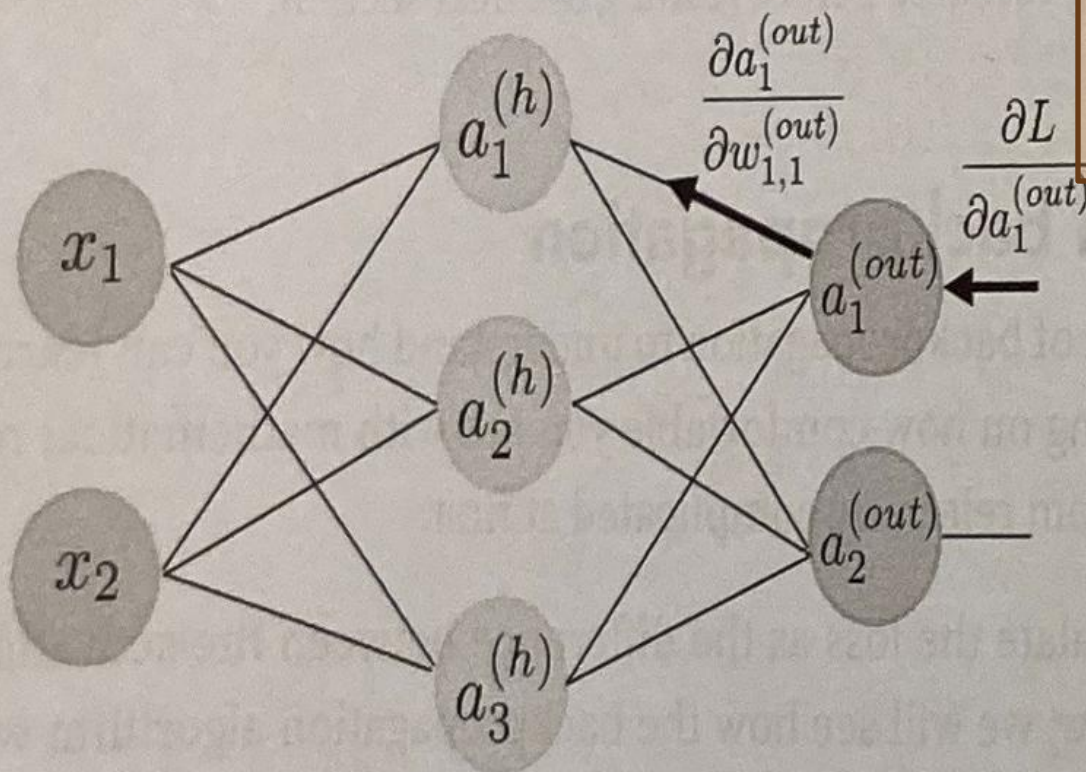
$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

The next term is the derivative of the logistic sigmoid activation function that we used in the output layer:

$$\begin{aligned} \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} &= \frac{\partial}{\partial z_1^{(out)}} \frac{1}{1 + e^{z_1^{(out)}}} = \dots = \left(\frac{1}{1 + e^{z_1^{(out)}}} \right) \left(1 - \frac{1}{1 + e^{z_1^{(out)}}} \right) \\ &= a_1^{(out)} (1 - a_1^{(out)}) \end{aligned}$$



$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

GIVEN:

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$$

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

Lastly, we compute the derivative of the net input with respect to the weight:

$$\frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}} = \frac{\partial}{\partial w_{1,1}^{(out)}} a_1^{(h)} w_{1,1}^{(out)} + b_1^{(out)} = a_1^{(h)}$$

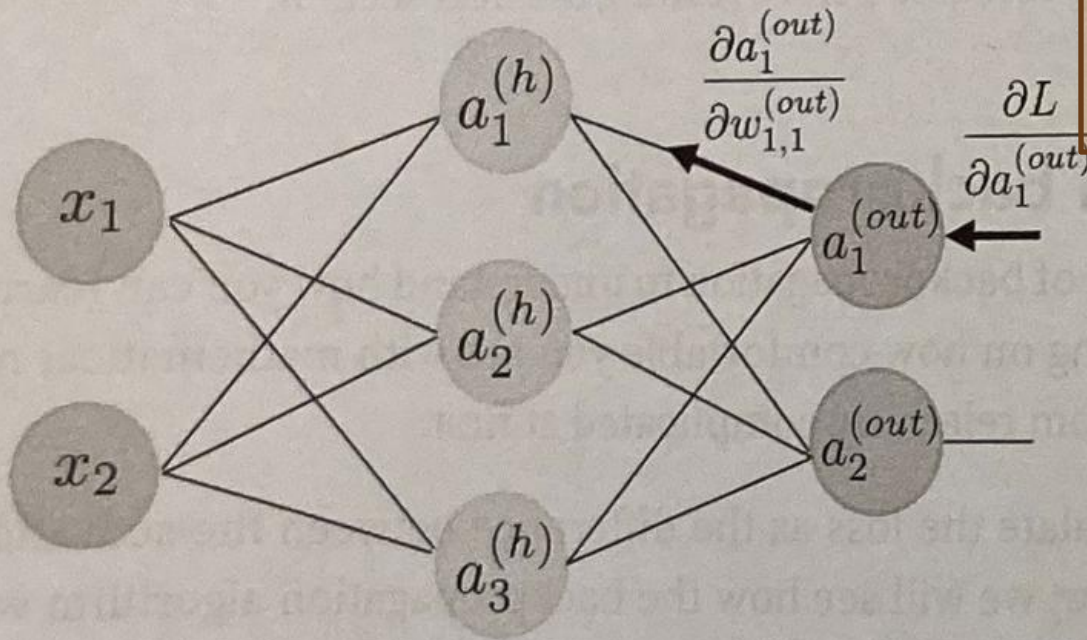
Python
Machine
Learning

Machine Learning and Deep Learning
with Python, scikit-learn, and TensorFlow 2

Third Edition - Includes TensorFlow 2,
GANs, and Reinforcement Learning

Sebastian Raschka
& Vahid Mirjalili

Packt



GIVEN:

$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$$

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

Putting all of it together, we get the following:

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}} = 2(a_1^{(out)} - y) \cdot a_1^{(out)}(1 - a_1^{(out)}) \cdot a_1^{(h)}$$

We then use this value to update the weight via the familiar stochastic gradient descent update with a learning rate of η :

$$w_{1,1}^{(out)} := w_{1,1}^{(out)} - \eta \frac{\partial L}{\partial w_{1,1}^{(out)}}$$

Python
Machine
Learning

Machine Learning and Deep Learning
with Python, scikit-learn, and TensorFlow 2

Third Edition - Includes TensorFlow 2,
GANs, and Reinforcement Learning

Sebastian Raschka
& Vahid Mirjalili

Packt

References

